CCF CSP 计算机软件能力认证

CCF CSP

第 41 次认证

时间：2026 年 3 月 29 日 13:30 ~ 17:30

题目名称	平衡数	机器人项目管理	进程通信	异或	旅游计划
题目类型	传统型	传统型	传统型	传统型	传统型
输入	标准输入	标准输入	标准输入	标准输入	标准输入
输出	标准输出	标准输出	标准输出	标准输出	标准输出
每个测试点时限	1.0 秒	1.0 秒	1.0 秒	6.0 秒	5.0 秒
内存限制	512 MiB	512 MiB	512 MiB	1024 MiB	1024 MiB
子任务数目	10	20	20	4	4
测试点是否等分	是	是	是	否	否

平衡数（balance）

【题目描述】

小 P 在学习了位运算后，定义了如下“平衡数”：

- 对于一个正整数 a，如果其二进制表示中 1 和 0 的个数相等，则称其为平衡数。

这里的二进制表示以 1 为最高位，即不考虑更高位用于补位的 0。例如，正整数 12 的二进制表示为 1100，包含两个 1 和两个 0。在实际存储中，更高位可能需要填 0 补位，如 00001100，在本题中无需考虑此种情况。

小 P 已经生成好了 n 个正整数 $a_1, a_2, \cdots, a_n$，试编程统计里面有多少个平衡数。

【输入格式】

从标准输入读入数据。

输入的第一行包含一个正整数 n，第二行包含空格分隔的 n 个正整数 $a_1, a_2, \cdots, a_n$。

【输出格式】

输出到标准输出。

输出一个整数，表示 $a_1, a_2, \cdots, a_n$ 中平衡数的个数。

【样例输入】

```
3
2 5 9
```

【样例输出】

```
2
```

【样例解释】

2 的二进制表示为 10，是平衡数；

5 的二进制表示为 101，不是平衡数；

9 的二进制表示为 1001，是平衡数。

【子任务】

50% 的测试点满足：$0 < a_i \leq 256$；

全部的测试点满足：$0 < n \leq 100$ 且 $0 < a_i \leq 10^9$。

机器人项目管理（crashing）

【题目描述】

小 P 计划招募 n 个机器人完成一个项目：每个机器人负责其中的一项任务，编号从 1 到 n，任务之间互不干扰。如果完成任务 i 的耗时为 t_i，则该项目总耗时为 $t_1 + t_2 + \cdots + t_n$。

作为项目管理者，小 P 可以用有限的预算为机器人们购买咖啡加油。其中负责任务 i 的机器人，最多可以喝 a_i 杯咖啡，从而将该任务耗时缩短 b_i（最终耗时即为 $t_i - b_i$）。

根据任务的特性和机器人的偏好，n 项任务可分为“灵活型”和“普通型”两类，详情参见附件资料（重要）。

已知小 P 可以为机器人们提供最多 m 杯咖啡，试计算完成整个项目的最短时间。

【输入格式】

从标准输入读入数据。

输入的第一行包含空格分隔的两个正整数 n 和 m，分别表示任务数量和咖啡数量。

接下来 n 行，每行包含空格分隔的四个整数 o_i、t_i、a_i 和 b_i，表示一个任务。其中 $o_i \in \{0, 1\}$ 表示任务类别，$o_i = 0$ 表示灵活型、$o_i = 1$ 表示普通型；其余变量含义如上所述，输入数据保证 $t_i > b_i$，即缩短后的耗时仍大于零。

【输出格式】

输出到标准输出。

输出一个实数，表示完成整个项目的最短时间。

【样例 1 输入】

```
3 5
0 2 3 1
0 3 4 2
0 4 5 2
```

【样例 1 输出】

```
6.6
```

【样例 1 解释】

三个任务均为灵活型，初始总耗时为 $2+3+4=9$。最优方案为：给任务二分配 4 杯咖啡，耗时缩短 2；给任务三分配 1 杯咖啡，耗时相应缩短 $\frac{2}{5}=0.4$。综上，完成整个项目最短时间为 $9-2-0.4=6.6$。

【样例 2 输入】

```
5 62
0 10 2 1
0 10 1 1
1 500 40 360
1 600 50 500
1 400 20 150
```

【样例 2 输出】

```
1008.5
```

【样例 2 解释】

初始总耗时为 1520。最优方案为：给任务三分配 40 杯咖啡，耗时缩短 360；给任务五分配 20 杯咖啡，耗时缩短 150；给任务一和二各分配 1 杯咖啡，耗时分别缩短 0.5 和 1。综上，完成整个项目最短时间为 $1520-360-150-0.5-1=1008.5$。

【子任务】

80% 的测试点满足：所有任务均为灵活型任务，且对于每个任务 i 有 $0<a_i\le 20$；

全部的测试点满足：$0<n\le 200$、$0<m\le 1000$，且对于每个任务 i 有 $0<a_i\le 100$、$0<b_i<t_i\le 10^4$。

【评分方式】

输出结果与标准答案相比绝对误差小于 0.001 即可，推荐保留六位小数输出结果。

进程通信（communicate）

【题目背景】

某一天，西西艾弗岛上的居民们迎来了一个天大的好消息：他们终于有了自己的操作系统。这个操作系统有一个特性：其内部每个进程都具有动态链接接口并对外广播信息的功能。小 C 的团队负责开发该功能与内存间交互的必要环节，同时维护支持内存调度与分配的子系统。

这套系统刚开发完毕，运行稳定性未知，于是小 C 团队邀请了你来当模拟测试员。他们希望你能够帮忙模拟出这套系统的运作流程，对于各种可能情况给出其具体行为，从而帮助他们完善这套系统。由于你们的合作刚处于起步阶段，小 C 为你提供了一个简化后的模型。

初始建构：

首先认为有一段存储容量为 10^{100}（保证足够大）的初始为空的全局内存，内存地址从左到右依次编号 $0, 1, 2 \cdots$。为了简化模型，视这段内存的每一个字节可以存储一个对象，且我们并不关心对象具体内容。因此本题中只需用 $e_i \in \{0, 1\}, x_i \in \{0, 1\}$ 两个状态变量代表 i 号地址处**是否被占用**以及**是否有对象被存储**；其中 $e_i = 1$ 代表地址 i 被占用，$e_i = 0$ 代表未被占用；$x_i = 1$ 代表有对象存储，$x_i = 0$ 代表没有。初始时所有 e_i, x_i 均为 0。

接下来考虑有 n 个准备进行对象传输的进程，依次编号 $1, 2, \cdots, n$，一开始没有任何接口与它们对接。初始状态下系统中各对象的逻辑关系可以用下图表示（例 $n = 2$）：

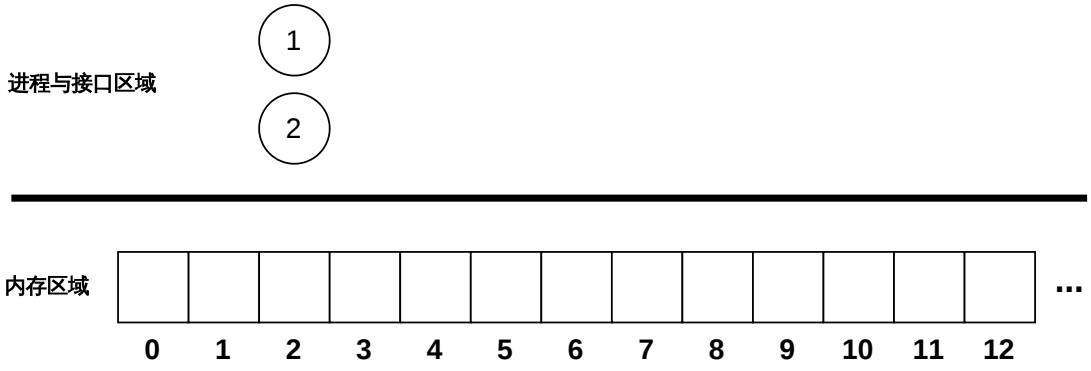

图 1: 初始化公用内存与进程

通信行为：

将系统内各对象的行为以操作的形式封装，接下来定义小 C 的模型中所有可能发生的操作：

`new p L`：建立一个只负责接收进程 p 所发出对象的新接口，同时建立在进程 p 与该接口之间进行消息传递的队列，队列存储容量为 L。

- 设在该操作之前进程 p 共对接了 k 个接口，新建的接口会被编号为 $k+1$。注意，新建的接口与队列具有对应关系，且它们均独属于进程 p。
- 接下来分配器会响应，根据**最优适应原则**存储队列。具体来说：
  - 分配器首先会识别内存中所有**极长的未被占用的连续地址段**，它们从左到右可以写成若干个非空区间的形式 $[l_1, r_1], [l_2, r_2], \cdots, [l_{t-1}, r_{t-1}], [l_t, r_t]$。其中 $r_t = 10^{100}$。从左到右第 i 段的长度为 $r_i - l_i + 1$。
  - 接下来它会在这些段中寻找**长度 $\geq L$ 且尽可能短**的段（若有多个满足条件的段，则取左端点 l 最小的段）$[l, r]$；
  - 将连续地址段 $[l, l+L-1]$ 分配给接口 $k+1$，用于存储队列中的信息。这段长度为 L 的内存空间每个位置此刻起均视为被占用，也即 $\forall j \in [l, l+L-1]$，令 $e_j = 1$。

下例中，进程 $1, 2$ 在操作执行前均已对接各自的 1 号接口，分别占用内存区域 $[2, 4]$ 与 $[9, 11]$；进行一次 `new 1 3` 操作后，进程 1 对接了自身的 2 号接口，并在内存区域 $[5, 7]$ 建立了对应的队列：

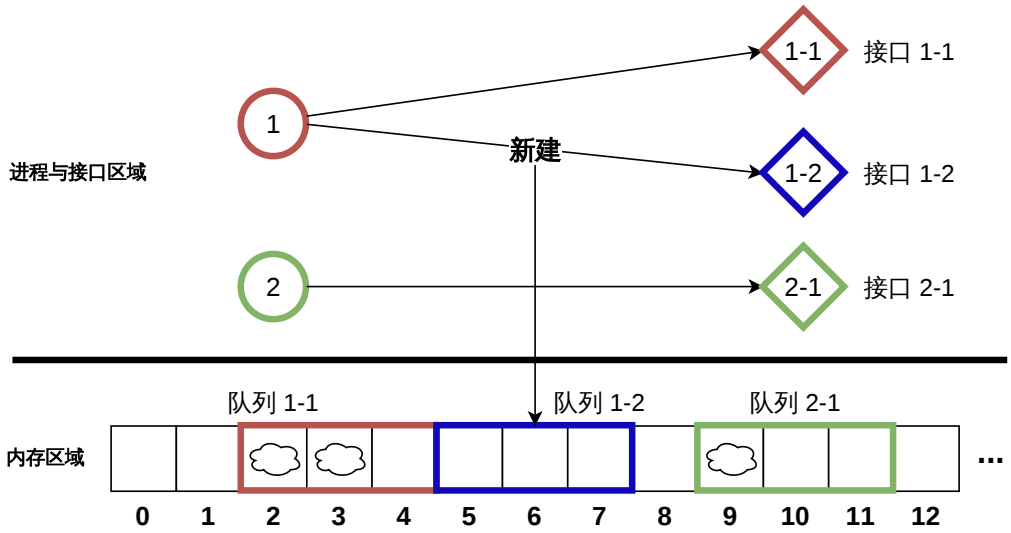

图 2: 新建操作示例

`send p`：进程 p 同时向所有其对接的接口发送一个对象。具体来说，设 p 对接了 k 个接口，则 $\forall i \in [1, k]$，均进行如下操作：

- 找到与 p 对接的编号为 i 的接口对应的队列，其在内存中占用的地址区间为 $[a, b]$。

- 如果队列为空（即接口建立以来进程 p 从未向该接口发送过任何对象），则将对象存储在 a 处，令 $x_a = 1$。
- 否则进程 p 向该接口发送过至少一个对象。记**最近一次**发送时，对象被存储在了位置 $t \in [a, b]$ 处。若 $t < b$，则该次发送将对象存储在地址 $t+1$ 处，令 $x_{t+1} = 1$；否则 $t = b$，此时将对象存储在地址 a 处，令 $x_a = 1$。注意该操作**可能不对某个地址的属性 x 造成改动**。

在上例的基础上，进行一次 `send 1` 操作，1 号进程会在两个队列分别占有的 $4, 5$ 号地址处各存储一个对象：

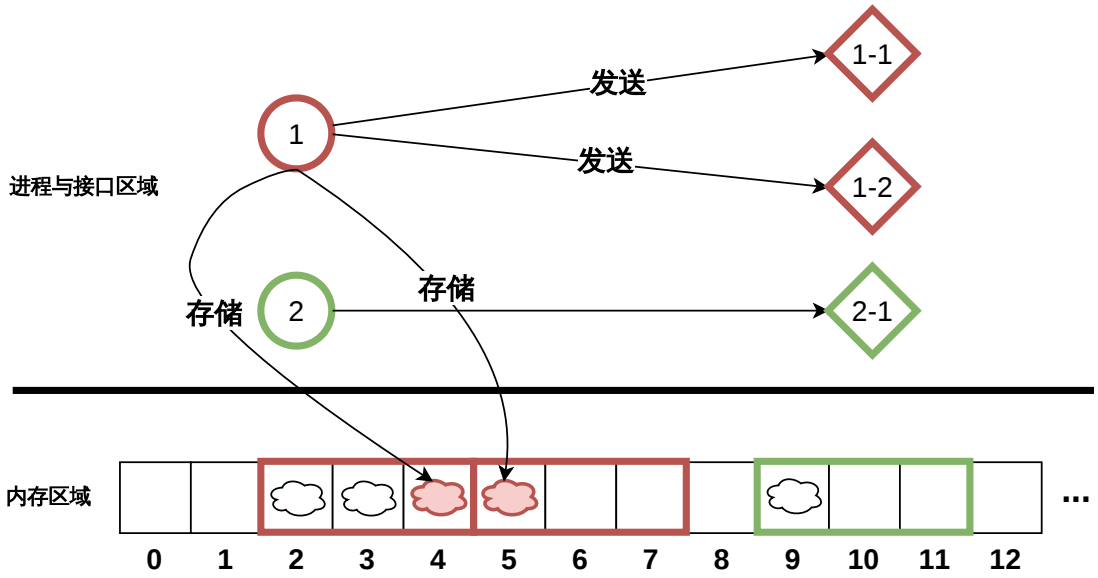

图 3: 发送操作示例

额外说明：如果在此之后连续执行 4 次 `send 2` 操作，则队列 $2-1$ 会依次在地址 $10, 11, 9, 10$ 存储对象。

`delete p i`：设进程 p 对接了 k 个接口，该操作保证 $1 \le i \le k$。该操作使编号为 i 的接口及队列均被删除，删除时遵循如下流程：

- 找到与进程 p 对接的编号为 i 的接口对应的队列，其在内存中占用的地址区间为 $[a, b]$。
- $\forall i \in [a, b]$，将 e_i 与 x_i 均置为 0，视为**删除所有存储在其中的对象，并将各个地址的占用状态取消**。
- 将与进程 p 所对接的其余编号大于 i 的接口的编号均减去 1，接口与队列的对应关系不变。例如一次操作前进程 p 对接了 4 个接口，则通过操作删除 2 号接口及队列后，原先编号为 $3, 4$ 的接口与队列的编号会被依次更新为 $2, 3$。

在上例的基础上，进行一次 `delete 1 1` 操作，则 1 号进程原先对应的 1 号队列及其内部的对象均会被删除：

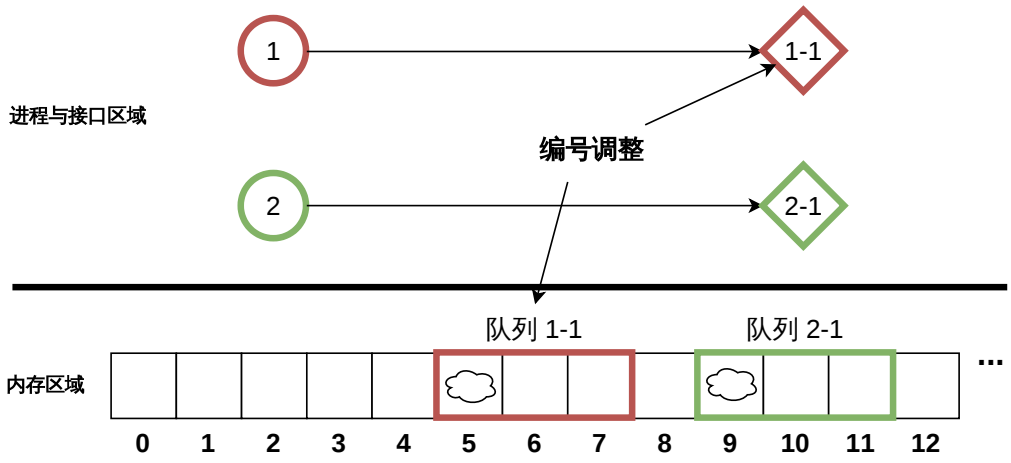

图 4: 删除操作示例

以上便是小 C 给出的简化模型的所有基本事件定义。

【题目描述】

现在小 C 给出了一个长度为 q 的操作序列，每个操作都为上述三种之一。你需要严格按照顺序模拟这些操作的运行。

小 C 为了检查模拟程序运行的过程是否正确，要求你进行如下反馈输出：

- 每次执行完 `new p L` 操作，你需要输出进程 p 所对接的新接口相应队列在内存中存储的地址。设其存储在区间 $[a, b]$，你只需输出 a。
- 每次执行完 `send p` 操作，在进程 p 向其对接的 k 个接口均发送一个对象后，你需要输出该操作中 k 个新发送的对象所存储的地址的和。

为了让你们之间的合作能更进一步，请你完成小 C 的模拟任务吧！

【输入格式】

从标准输入读入数据。

第一行用空格隔开的两个整数 n, q，依次代表进程数量与操作数量。

接下来 q 行，每行为一个操作。操作格式如题目背景所描述。单词与数字、数字与数字之间由一个空格隔开。

【输出格式】

输出到标准输出。

请你对于每一个 new 操作与 send 操作，按照题目要求输出一行一个整数。

【样例 1 输入】

```
2 13
new 1 2
new 1 3
send 1
delete 1 1
new 1 4
send 1
new 2 3
send 2
delete 1 2
new 1 3
send 1
delete 1 1
send 1
```

【样例 1 输出】

```
0
2
2
5
8
9
9
5
9
6
```

【样例 1 解释】

读入数据自初始状态起进行完前 9 次操作后变为操作解释中的状态。

每次新建操作的初始地址依次为 $0, 2, 5, 9, 5$；
每次发送操作所存储的所有对象所在地址和依次为 $2, 8, 9, 9, 6$。

【样例 2】

见题目目录下的 ***2.in*** 与 ***2.ans***。

【样例 3】

见题目目录下的 ***3.in*** 与 ***3.ans***。

【子任务】

保证操作均有意义。即不会建立空队列，不会在进程 x 没有对接任何接口时发送对象，不会删除不存在的队列与接口。特别注意：**向某长度为 l 的队列发送超过 l 个对象的行为是有意义的。**

记所有 `new` 操作中的参数 L 的最大值为 L_m。

- 对于前 40% 的测试数据，保证不存在 `delete` 操作；
- 对于前 80% 的测试数据，保证 $L_m \le 10, q \le 800$；

对于所有测试数据，保证 $1 \le n \le 100, 1 \le q \le 8000, 1 \le L_m \le 5 \times 10^5$，除操作名称外所有输入数据均为非负整数。

异或（xor）

【题目描述】

类似二进制下的异或操作，小 C 定义了一种 k 进制下的异或运算 $\oplus_k$。

对于任意非负整数 a, b 定义 a 和 b 进行 k 进制下的**不进位**加法的结果为 $a \oplus_k b$。

形式化地，如果 $a = \sum_{i=0}^{n}(a_i \cdot k^i), b = \sum_{i=0}^{n}(b_i \cdot k^i)$，其中 a_i, b_i 是**小于** k 的非负整数，则 $a \oplus_k b = \sum_{i=0}^{n}(((a_i + b_i) \bmod k) \cdot k^i)$。

如计算 $11 \oplus_3 7$ 的过程如下：11 可表示为 $2 \times 3^0 + 0 \times 3^1 + 1 \times 3^2$，7 可表示为 $1 \times 3^0 + 2 \times 3^1 + 0 \times 3^2$，则 $11 \oplus_3 7 = ((2+1) \bmod 3) \times 3^0 + ((2+0) \bmod 3) \times 3^1 + ((0+1) \bmod 3) \times 3^2 = 0 \times 3^0 + 2 \times 3^1 + 1 \times 3^2 = 15$。

之后小 C 在此运算基础上对任意非负整数 n 定义了一个函数 f，具体如下：

$$f(n) = \begin{cases} 0 & (n = 0) \\ n \oplus_k f(n-1) & (n > 0) \end{cases}$$

现在小 C 有一个非负整数序列 $a_1, a_2, \cdots, a_n$ 和**全局给定**的**奇数** k。小 C 希望对这个序列进行 m 次操作：

- 操作一给出三个参数 l, r, v，表示将下标在区间 $[l, r]$ 内的 a_i 修改为 $a_i \oplus_k v$。
- 操作二给出两个参数 l, r，表示查询 $f(a_l) \oplus_k f(a_{l+1}) \oplus_k \cdots \oplus_k f(a_r)$。

【输入格式】

从标准输入读入数据。

第一行给出三个正整数 n, m, k。

接下来一行给出 n 个非负整数，表示序列 a。

接下来 m 行每行给出三个或者四个非负整数，表示 m 次操作。

每行第一个数字为操作的类型 t。

- 如果 t 为 1，则该操作为操作一，接下来给出三个非负整数 l, r, v 作为该操作的参数。
- 如果 t 为 2，则该操作为操作二，接下来给出两个正整数 l, r 作为该操作的参数。

【输出格式】

输出到标准输出。

对于每个操作二，输出单独的一行，包含一个非负整数，为该操作的答案。

【样例 1 输入】

```
5 4 11
13 17 14 19 15
1 1 3 9
2 1 2
1 3 5 6
2 4 5
```

【样例 1 输出】

```
76
50
```

【样例 1 解释】

- 第一个操作结束之后，序列 a 为：$[11, 15, 12, 19, 15]$。
- 第二个操作的答案为 $f(a_1) \oplus_{11} f(a_2) = 11 \oplus_{11} 65 = 76$。
- 第三个操作结束之后，序列 a 为：$[17, 21, 18, 14, 21]$。
- 第四个操作的答案为 $f(a_4) \oplus_{11} f(a_5) = 50 \oplus_{11} 0 = 50$。

【样例 2】

见题目目录下的 ***2.in*** 与 ***2.ans***。
该样例满足子任务一的限制。

【样例 3】

见题目目录下的 ***3.in*** 与 ***3.ans***。
该样例满足子任务二的限制。

【样例 4】

见题目目录下的 ***4.in*** 与 ***4.ans***。
该样例满足子任务三的限制。

【样例 5】

见题目目录下的 ***5.in*** 与 ***5.ans***。
该样例满足子任务四的限制。

【子任务】

本题采用捆绑测试，你只有通过一个子任务中的所有测试点才能得到该子任务的分数。

对于所有数据满足：$1 \le n \le 5 \times 10^5, 1 \le m \le 10^4, 11 \le k \le 10^9$ 且 k **为奇数**。对于每个操作保证 $t \in \{1, 2\}, 1 \le l \le r \le n$。对于任意时刻，保证 a_i 非负。对于所有操作一，保证参数 v 非负。记所有时刻序列中的元素 a_i 和所有操作一中的参数 v 的最大值为 R，则保证 $R \le 10^{12}$。

- 子任务一（20 分）：保证 $n, m \le 10^3$ 且 $R \le 10^6$。
- 子任务二（30 分）：保证 $n, m \le 10^3$。
- 子任务三（20 分）：保证 a_i 和操作一的参数 v 均是 k 的倍数。
- 子任务四（30 分）：无特殊限制。

【提示】

子任务三可能有助于你更好地探索函数 f 的相关性质。

旅游计划（travel）

【题目背景】

小 z 想要环游世界，现在他来到了 A 国。

【题目描述】

A 国有 n 座城市（编号为 1 到 n），由 $n-1$ 条**双向**道路相连，保证任意两座城市之间都由道路**直接或间接**相连。

小 z 有 m 个开车旅游计划，每个旅游计划用 (s_i, t_i) 描述，表示从 s_i 号城市出发**沿最短路径**去到 t_i 号城市。

小 z 发现 A 国的基建水平非常糟糕，经过观察：

- 初始**所有**道路都没有被翻修。
- 经过一条没有翻修的道路会爆一个轮胎。
- 小 z 有一个备用轮胎。如果备胎是好的，小 z 可以将损坏的轮胎更换后继续移动；如果备胎是已经更换下来的轮胎，车将无法移动。
- 有 k 座城市里有维修站，小 z 可以在那里把所有轮胎修好。
- 初始所有轮胎都是好的。

对于一个旅游计划 (s_i, t_i)，如果车能从 s_i 成功开到 t_i，那么旅游计划是可行的。

给定 q 次事件：

- `1 u v`，连接 u, v 的道路进行了翻修，保证这条路存在且之前没有被翻修。
- `2`，询问有多少旅游计划可行。

小 z 希望能够及时得到信息，所以本题**强制在线**，你需要对于每个询问事件立即回答。

【输入格式】

从标准输入读入数据。

记录 $lastans$ 为上一次询问事件的答案，初始 $lastans = 0$。

读入一行两个整数 n, X。X 是强制在线的加密参数。

接下来 $n-1$ 行，每行读入两个整数 u, v，表示城市 u 和 v 之间有一条道路。

读入一行一个整数 k。

接下来一行，读入 k 个整数，表示有维修站的城市，保证这 k 个城市互不相同。

读入一行一个整数 m。

接下来 m 行，第 i 行读入两个整数 s_i, t_i，表示有一个从 s_i 到 t_i 的旅游计划，保证 $s_i \neq t_i$。

读入一行一个整数 q。

接下来 q 行，每行先读入一个正整数 op：

- 若 $op = 1$，表示发生 1 事件，接下来读入两个整数 u', v'，实际的 $u = u' \oplus (X \times lastans), v = v' \oplus (X \times lastans)$。$\oplus$ 表示异或操作。
- 若 $op = 2$，表示发生 2 事件。

事件含义同上文描述。

【输出格式】

输出到标准输出。

对于每一个询问事件，输出一行一个整数，表示有多少个旅游计划可行。

【样例 1 输入】

```
5 0
1 2
3 4
1 5
2 3
5
4 3 1 2 5
3
5 2
2 5
1 3
7
2
1 2 3
1 1 5
1 1 2
1 3 4
2
2
```

【样例 1 输出】

```
3
3
3
```

【样例 2 输入】

```
5 0
1 2
3 4
2 3
3 5
1
1
6
5 1
5 1
2 3
3 5
5 2
2 4
8
2
2
2
2
1 3 5
1 2 3
2
1 1 2
```

【样例 2 输出】

```
2
2
2
2
6
```

【样例 3】

见题目目录下的 ***3.in*** 与 ***3.ans***。

【样例 4】

见题目目录下的 ***4.in*** 与 ***4.ans***。

【样例 5】

见题目目录下的 ***5.in*** 与 ***5.ans***。

【样例 6】

见题目目录下的 ***6.in*** 与 ***6.ans***。

【子任务】

本题采用捆绑测试，你只有通过一个子任务中的所有测试点才能得到该子任务的分数。

所有数据满足，$2 \le n \le 1 \times 10^5$，$1 \le m \le 1 \times 10^5$，$1 \le q \le 2 \times 10^5$，$1 \le k \le 20$，$X \in \{0, 1\}$，保证输入数据合法。

- 子任务一（10 分）：$1 \le n, m, k, q \le 10$。
- 子任务二（20 分）：$1 \le n, m, k, q \le 100$。
- 子任务三（30 分）：$X = 0$，即询问可以离线处理。
- 子任务四（40 分）：$X = 1$，即询问全部强制在线。